SEM Dendrite Segmentation: Comparative Technical Report

Nadav Robinson Yuval Cohen

March 12, 2026

Abstract

This report summarizes pixel-level semantic segmentation of lithium dendrites in SEM images using two pipelines: (1) a deterministic classic morphology pipeline and (2) a YOLO segmentation model. The evaluation set includes 14 labeled images. Average Dice scores are Classic=0.845 and YOLO=0.449. The report includes quantitative metrics, method comparison tables, and visual success/failure analysis.

Methodology

Classic Pipeline (`scripts/classic_scripts/classic_pipeline.py`)

The classical pipeline is deterministic and is executed in fixed stages:

1. **Input and cleanup:** each SEM image is loaded in grayscale, then the bottom instrument region (scale bar / metadata strip) is removed using `remove_scale_bar`.
2. **Pre-processing (`preprocess`):**
   - `normalize_histogram`: linear stretch to full $[0, 255]$, reducing exposure variation across images.
   - `apply_clahe`: local contrast enhancement for weak dendrite visibility.
   - `apply_bilateral_filter`: denoising while preserving thin branch edges.
3. **Segmentation (`segment_adaptive`):** adaptive Gaussian thresholding is applied with local neighborhood size `ADAPTIVE_BLOCK_SIZE` and bias `ADAPTIVE_C`. This is preferred over global thresholding due to non-uniform illumination in SEM images.
4. **Post-processing (`postprocess`):**
   - `morphological_reconstruction`: geodesic reconstruction after erosion to suppress noise while preserving dendrite geometry.
   - `apply_closing`: fills small holes and improves branch continuity.
   - `remove_small_components` (enabled by profile): removes isolated small blobs unless proximity logic indicates likely branch continuation.
5. **Branch separation (`separate_branches`):** distance transform + marker-based watershed separates touching dendrites by cutting watershed boundaries.
6. **Skeletonization and outputs:** `skeletonize_mask` extracts a 1-pixel centerline from the final mask. Masks/skeletons are restored to the original canvas size, and intermediate artifacts are saved (`01_original` ... `12_preview`) for traceability.

Classic Parameter Profiles Used in This Project

The orchestrator (`scripts/run_all.py`) applies profile-specific parameters by subset (`dataset/easy` vs `dataset/hard`) through `apply_parameter_profile`.

Common baseline parameters (shared unless overridden):

- `CLAHE_CLIP_LIMIT` = 1.0
- `CLAHE_TILE_SIZE` = 8
- `BILATERAL_D` = 9
- `BILATERAL_SIGMA_COLOR` = 50
- `BILATERAL_SIGMA_SPACE` = 50
- `ADAPTIVE_BLOCK_SIZE` = 67
- `ADAPTIVE_C` = -12
- `EROSION_KERNEL_SIZE` = 5
- `EROSION_ITERATIONS` = 1
- `CLOSING_KERNEL_SIZE` = 3
- `DISTANCE_THRESHOLD` = 0.07
- `COMPONENT_PROXIMITY_RADIUS` = 30
- `COMPONENT_PROXIMITY_RATIO` = 0.5

Easy profile (`dataset/easy`) overrides:

- `RUN_SMALL_REMOVAL` = True
- `MIN_COMPONENT_AREA` = 75

Hard profile (`dataset/hard`) overrides:

- `RUN_SMALL_REMOVAL` = True
- `MIN_COMPONENT_AREA` = 450
- `COMPONENT_PROXIMITY_RADIUS` = 50

Deep Learning Pipeline (`scripts/yolo_scripts/yolo_pipeline.py`)

The deep model is Ultralytics YOLO segmentation (`yolo11x-seg.pt`) with transfer learning.

1. **Dataset normalization:** `ensure_three_channel_dataset` converts grayscale / single-channel / BGRA images to 3-channel BGR PNG and clears stale cache files, ensuring compatibility with pretrained YOLO backbones.
2. **Training setup used:** from the saved run arguments (`output/yolo/train/dendrite_seg/args.yaml`): model=`yolo11x-seg.pt`, epochs=100, image size=640, batch=8, initial learning rate $lr0 = 0.001$, patience=0, freeze=0, workers=0.
3. **Inference and mask extraction:** inputs are normalized by `_prepare_image_for_yolo` (channel fix + dtype normalization), then `predict_batch` runs inference with confidence threshold `conf`=0.25. All instance masks are resized to image resolution and merged with logical OR ($mask > 0.5$) to produce a final binary $\{0, 255\}$ mask per image.

Evaluation Metrics

Both pipelines are evaluated against ground truth using Dice, IoU, Precision, and Recall. Failure analysis is then derived from metric patterns (over-segmentation vs under-segmentation vs fundamental mismatch).

Results and Discussion

The discussion below focuses on operational behavior: stability, typical error modes, and deployment implications.

Average Metrics

Method	Dice	IoU	Precision	Recall
Classic	0.845	0.787	0.788	0.932
YOLO	0.449	0.302	0.335	0.709

Table 1: Average segmentation metrics across available predictions.

Classic clearly outperforms YOLO in this run (Dice 0.845 vs 0.449 and IoU 0.787 vs 0.302), indicating better overlap quality with masks. Classic recall is also consistently high, while YOLO shows weaker precision and lower overlap consistency, reflecting frequent over-segmentation or domain mismatch.

Per-Image Comparison

Image	C-Dice	C-IoU	C-Prec	C-Rec	Y-Dice	Y-IoU	Y-Prec	Y-Rec
2e-9_100s_002	0.923	0.857	0.857	1.000	0.579	0.407	0.409	0.991
2e-9_100s_014	0.931	0.872	0.872	1.000	0.655	0.487	0.560	0.789
2e-9_100s_017	0.699	0.538	0.538	1.000	0.103	0.055	0.066	0.241
2e-9_100s_019	0.997	0.995	0.995	1.000	0.596	0.424	0.454	0.865
70nm_diameter_100nm_pitch_008	0.927	0.863	0.863	1.000	0.438	0.280	0.302	0.798
70nm_diameter_100nm_pitch_012	0.919	0.850	0.850	1.000	0.523	0.354	0.399	0.758
70nm_diameter_100nm_pitch_022	0.995	0.991	0.991	1.000	0.454	0.293	0.374	0.577
70nm_diameter_100nm_pitch_031	1.000	1.000	1.000	1.000	0.359	0.218	0.272	0.527
70nm_diameter_100nm_pitch_033	0.024	0.012	0.017	0.043	0.065	0.034	0.039	0.194
Ag_1e-8_008	1.000	1.000	1.000	1.000	0.436	0.278	0.325	0.660
Ag_1e-9__02_016	0.906	0.828	0.828	1.000	0.560	0.389	0.432	0.796
Ag_2e-9_013	0.760	0.612	0.612	1.000	0.518	0.350	0.351	0.994
Ag_2e-9_014	0.753	0.603	0.603	1.000	0.479	0.315	0.316	0.994
Ag_40nm_pitch_014	1.000	1.000	1.000	1.000	0.514	0.346	0.395	0.736

Table 2: Per-image metric comparison (first 14 rows). Full table is exported to CSV.

The per-image table shows that Classic remains strong on most samples, including several near-perfect cases (Dice close to 1.0). YOLO performance is more variable: moderate on some images but significantly degraded on others, especially where texture statistics differ from the training distribution. The most severe outlier (`70nm_diameter_100nm_pitch_033`) affects both methods and should be treated as a hard-failure sample in downstream workflows.

Failure Analysis

Number of failures: 8.

Image	Method	Dice	Precision	Recall	Estimated Cause
70nm_diameter_100nm_pitch_033	Classic	0.024	0.017	0.043	Zoomed-out view with multi-layered noise; dendrite localization is visually challenging even for humans.
70nm_diameter_100nm_pitch_033	YOLO	0.065	0.039	0.194	Same cause as the Classic.
2e-9_100s_017	YOLO	0.103	0.066	0.241	Severe, consistent noise blends seamlessly with dendrites, causing the model to merge background and foreground. Highly challenging sample.
70nm_diameter_100nm_pitch_031	YOLO	0.359	0.272	0.527	Same cause as 2e-9_100s_017.
Ag_1e-8_008	YOLO	0.436	0.325	0.660	Over-segmentation.
70nm_diameter_100nm_pitch_008	YOLO	0.438	0.302	0.798	Over-segmentation.
Ag_2e-9_014	YOLO	0.479	0.316	0.994	Over-segmentation.

Table 3: Failure characterization based on precision/recall patterns.

Failure patterns are asymmetric between methods. Most flagged YOLO failures combine low precision with moderate-to-high recall, which matches over-segmentation (false-positive expansion into background structures). Classic has one major failure on the same outlier image, suggesting a true data difficulty case rather than a single-model artifact. From a risk perspective, this supports Classic as baseline and YOLO as auxiliary signal rather than primary mask source in the current configuration.

Visual Analysis (Successes and Failures)

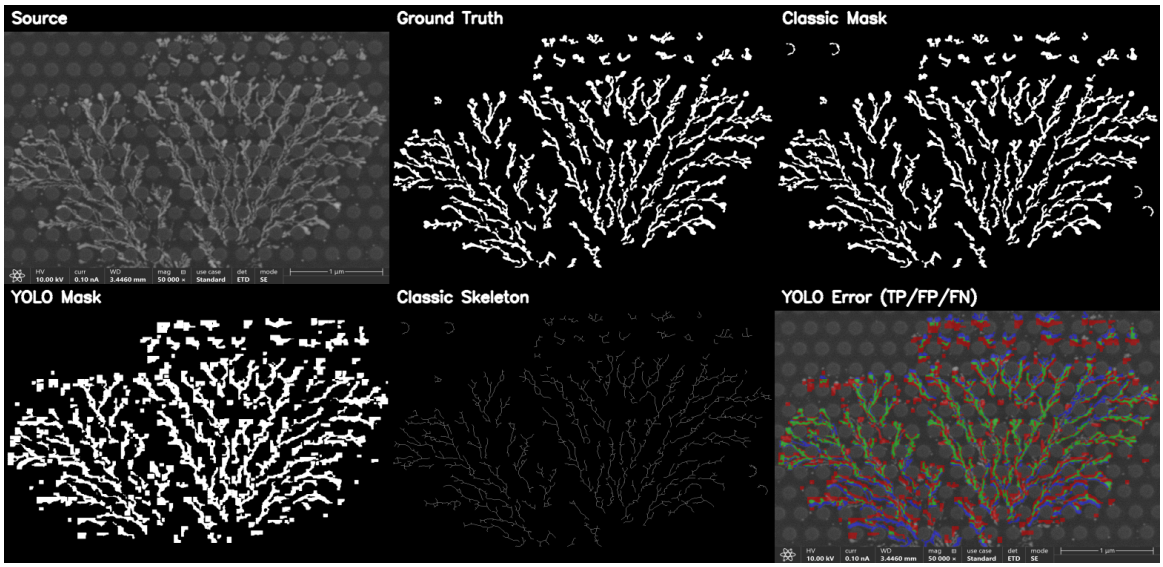

Figure 1: Success case: 70nm_diameter_100nm_pitch_022 (combined Dice=0.670)

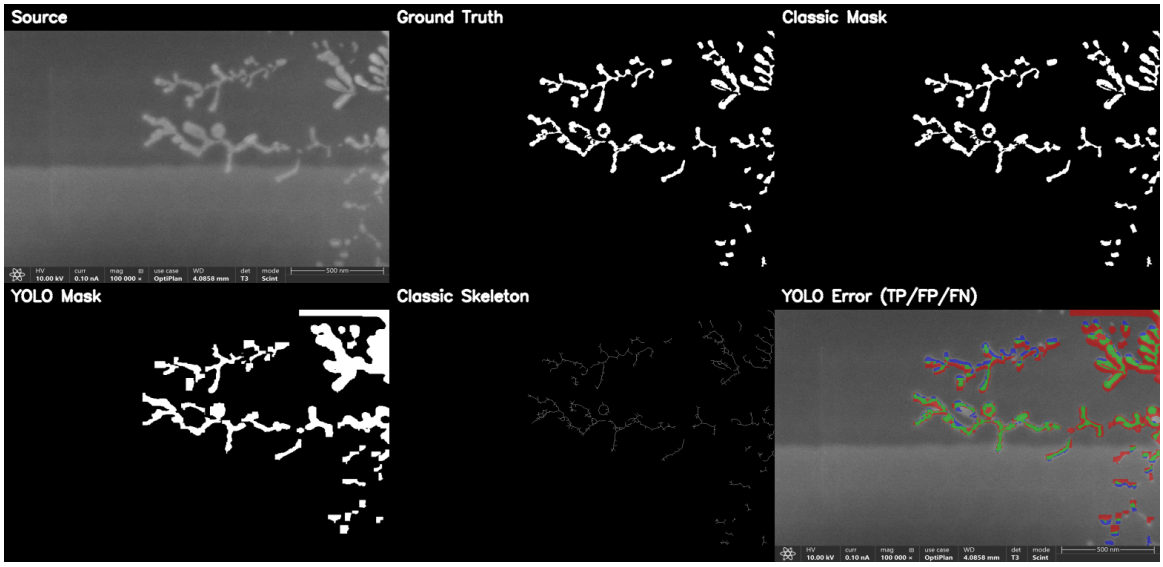

Figure 2: Success case: Ag_40nm_pitch_014 (combined Dice=0.749)

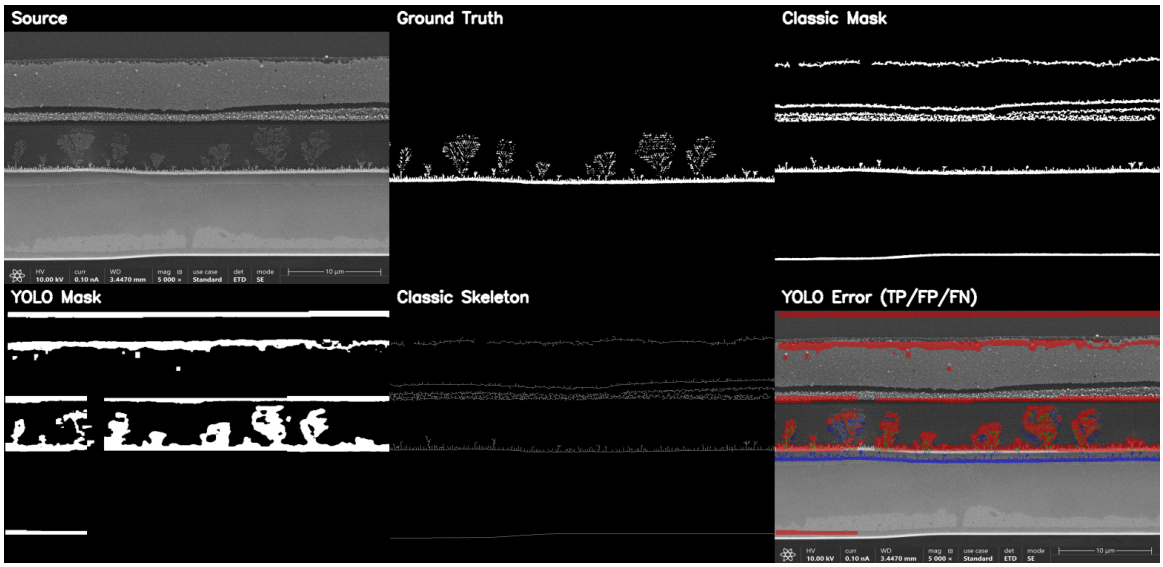

Figure 3: Failure case: 70nm_diameter_100nm_pitch_033 (combined Dice=0.015)

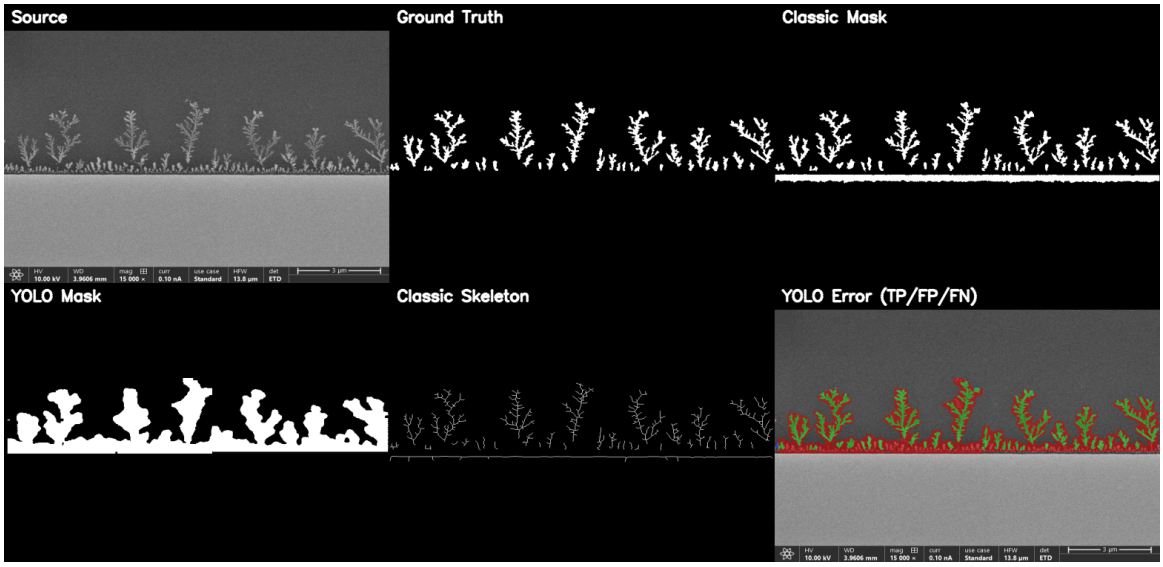

Figure 4: Failure case: Ag_2e-9_014 (combined Dice=0.470)

YOLO Training Diagnostics

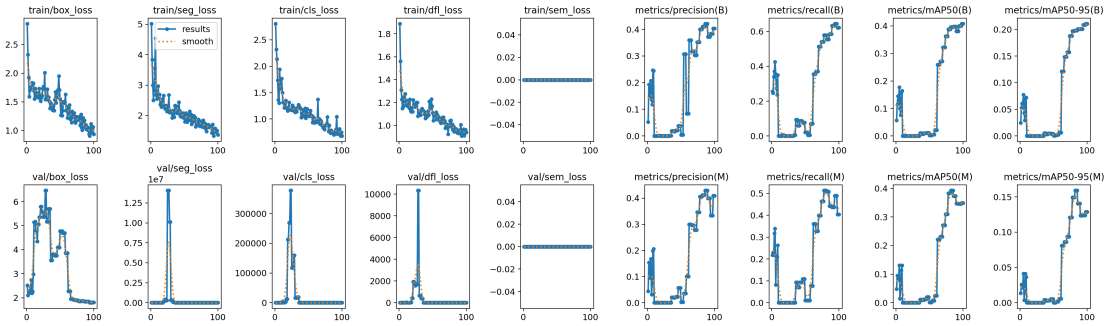

Figure 5: YOLO training metrics over epochs

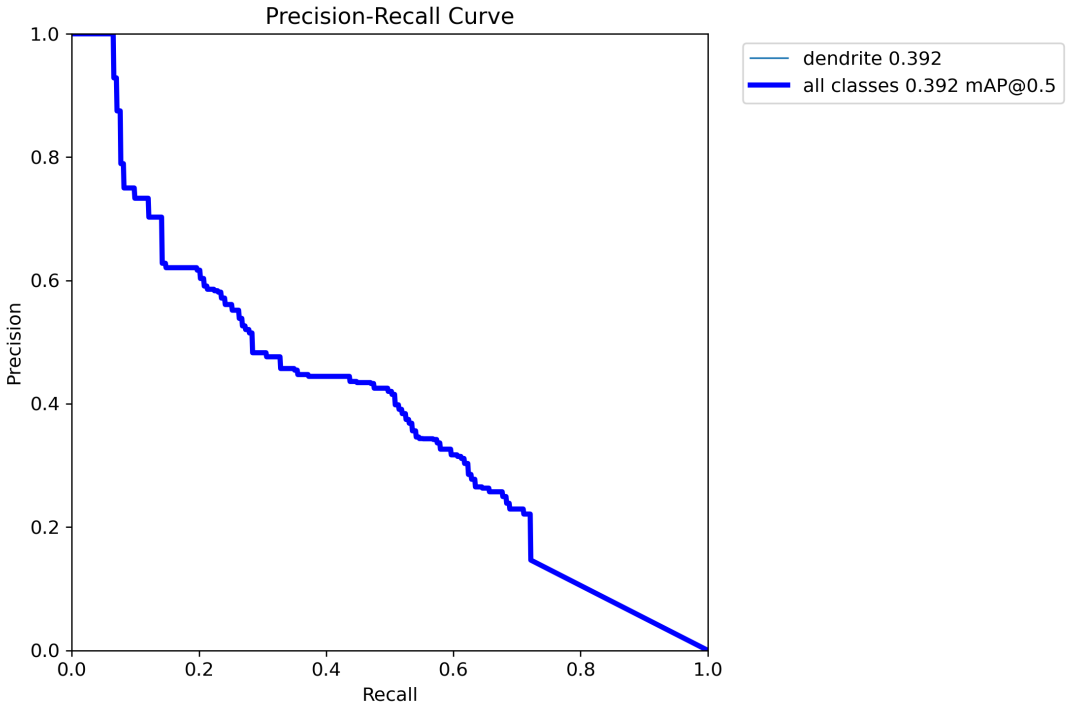

Figure 6: Mask precision-recall curve

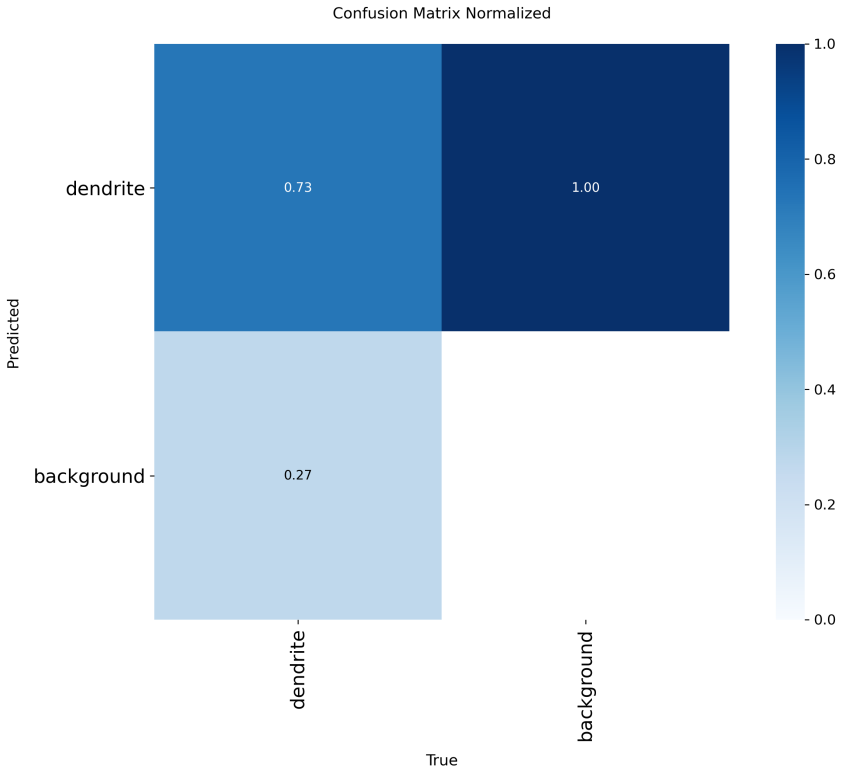

Figure 7: Normalized confusion matrix

Conclusions

For this dataset and preprocessing setup, the classic pipeline is the preferred primary method. Its average Dice is 0.845 versus 0.449 for YOLO, with consistently stronger IoU and precision-recall balance across most samples.

Which method to use in which scenario:

1. **Current SEM domain with limited retraining budget:** recommented to use **Classic pipeline** as default. It is more stable under the current image statistics and profile split (Easy/Hard), and it produced the strongest overall segmentation quality in this report.

2. **High-throughput deployment after domain-specific retraining:** use **YOLO** when enough representative labeled data is available and periodic retraining is feasible. In the present run, YOLO shows useful recall behavior but suffers from domain mismatch failures and over-segmentation in several hard cases.

3. **Outlier/severe-artifact samples (e.g., 70nm_diameter_100nm_pitch_033):** neither method is reliable enough alone; enforce **manual review/QA fallback**. These cases should be prioritized for additional labeling and targeted parameter or model updates.

Recommended operating policy: run Classic as the baseline automatic segmenter, keep YOLO as a secondary comparator, and trigger human validation when both methods disagree or low-confidence behavior is detected. This hybrid policy gives robust near-term results while preserving a path to future model-driven scaling.